
(Ne)Strukturirani podaci

— Problemi, slučajevi upotrebe, —
DataFrame API

Primjer obrade

- RDD API - oko 5.5 sekundi
- DataFrame API - oko 1.9 sekundi
- RDD API koristi reduceByKey dok DataFrame API primjer ima puno groupBy operacija koje bi trebale biti skupe
- Zašto je drugi primjer značajno brži?

Trenutni problemi?

- Kompleksan programski model
- Moramo paziti koje operacije koristimo
- Da li se desava shuffling ili ne?
- Idealan slučaj - programiranje upotrebom Spark-a bez razmišljanja o internom funkcionisanju
- Moguće uz određena ograničenja

Strukturirani i nestruktuirani podaci

Unstructured data

The university has 5600 students.
John's ID is number 1, he is 18 years old and already holds a B.Sc. degree.
David's ID is number 2, he is 31 years old and holds a Ph.D. degree. Robert's ID is number 3, he is 51 years old and also holds the same degree as David, a Ph.D. degree.

Semi-structured data

```
<University>
  <Student ID="1">
    <Name>John</Name>
    <Age>18</Age>
    <Degree>B.Sc.</Degree>
  </Student>
  <Student ID="2">
    <Name>David</Name>
    <Age>31</Age>
    <Degree>Ph.D. </Degree>
  </Student>
  ....
</University>
```

Structured data

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.

Nestruktuirani podaci

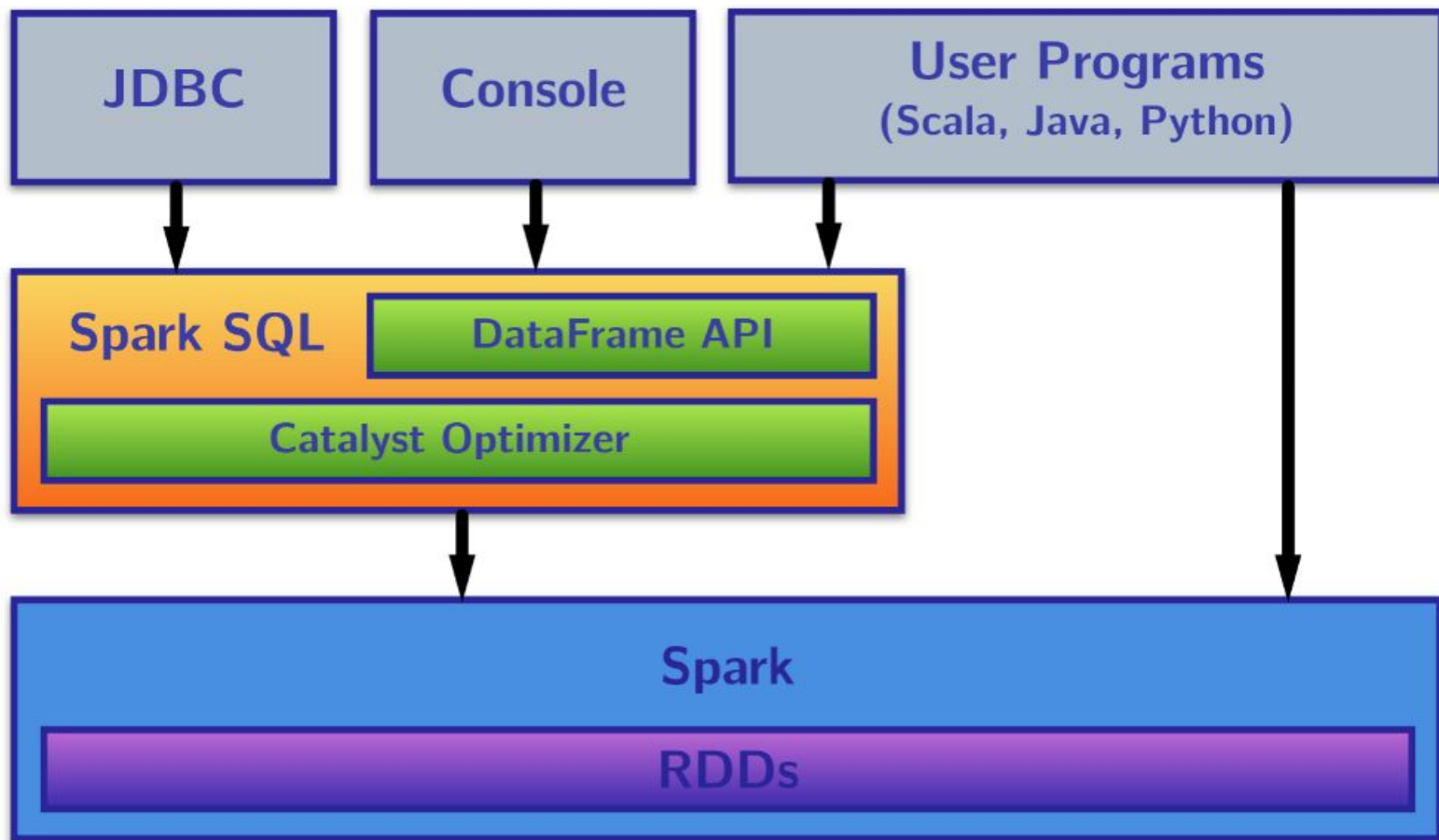
- RDD API
- Spark ne zna ništa o formatu podataka
- Spark vidi samo objekte - ne zna koje primitivne tipove ili podobjekte sadrže ti objekti
- Takođe spark ne zna ni sadržaj naših funkcija - funkcije vidljive isto samo kao neki vid objekata
- Optimizacija moguća samo na nivou globalnih Spark operacija - moramo razmišljati više o samom programskom modelu

Struktuirani podaci

- Podaci u tabelarnoj formi
- Spark zna tačno koji podaci postoje i koji su njihovi tipovi
- Skup operacija postaje ograničeniji - ne smijemo da razbijemo struktuiranost
- Omogućava Sparku da vrši puno agresivnije optimizacije
- Programski model postaje deklarativan
- Spark SQL API osnova - DataFrame glavna apstrakcija
- Ne razmišljamo više o objektima već o kolonama i redovima!
- Obavezna šema podataka!

DataFrame API

- RDD + Catalyst optimizer za strukturirane podatke
- Ograničeni skup [tipova](#)
- Najčešće operacije: select, agg, join, filter, groupBy (nije više loš)
- Čišćenje podataka jako bitno - stvarni skupovi podataka imaju puno null i nevalidnih vrijednosti - prave probleme u brojnim zadacima, posebno u MLu - koristimo drop i fill operacije
- Osnovni problem - neke zadatke ne možemo riješiti sa ograničenim skupom tipova i operacija



Catalyst optimizer

- Osnovni razlog bržeg rada DataFrame API-ja
- Poznaje:
 - šemu podataka
 - korištene tipove
 - implementaciju korištenih operacija
- Mogućnosti:
 - reordering operacija
 - smanjivanje podataka koji se trebaju čitati ili slati preko mreže
 - pruning particija - analiza operacija i izbacivanje particija koje nam nisu potrebne za operacije

Enkodovanje podataka

- Bitan aspekt performansi - koliko podataka može stati u memoriju i u kom djelu memorije se nalaze dati podaci
- Tungsten - Spark SQL off-heap data encoder
- Tipovi su ograničeni - moguće dodatne optimizacije sa aspekta memorije:
 - specijalizovani data encodери - više podataka može stati u memoriju (podaci usko zapakovani)
 - column-based encoder - podaci se čuvaju po kolonama (ne redovima) - puno brži rad operacija kao što su groupBy i lookup nad kolonama
 - off-heap - težnja da se što manje podataka čuva na heapu, smanjuje GC overhead